higher sensibly outperform a binary heap (at least in our example application).

Can we also implement a treap with a generic branching factor greater than 2?

Unfortunately, it's not that simple: first and foremost, we are using binary search trees, so, trees with a branching factor 2: if the heap's branching factor didn't match the BST, we would have a mess!

Then you might suggest to use ternary search trees, or their generalization: that, however, would make the rotation operations much more complicated, which means the code of the implementation would become terribly complicated and unclean (which likely also means slower!); moreover, we would have a harder time keeping the tree balanced, unless we use something like a 2-3 tree, but that's already guaranteed to be a balanced tree.

3.3.3 Implementing Search

Now that we have a better idea of how a treap is going to be represented in memory and how rotations work, we can move to the implementation of the main API's methods. You can also find a Java implementation of treaps in the book's repo on GitHub.

We can start from the search method, that's the easiest to describe: in fact, it's just the plain search method implemented in binary search trees: we traverse the tree from the root till we find the key we are looking for, or reach a leaf without finding it.

Like in plain BSTs, we only traverse one branch of each subtree, going left or right depending on how the target key compares to current node's key.